

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA
FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

TEMA:

Actividad Práctica:

ASIGNATURA:

BI

CURSO Y PARALELO:

6/1

DOCENTE:

ING. ANTHONY PACHAY ESPINOZA, MSC.

ESTUDIANTES:

JOSE DANIEL GONZALEZ COCHEA

1. Modelo dimensional

Arquitectura implementada: Dataset Kaggle → Limpieza y transformación → PostgreSQL → Modelo dimensional → Vistas / Funciones SQL → Aplicación analítica.

El modelo es una estrella con una tabla de hechos y tres dimensiones (tiempo, producto y cliente). Los datos crudos se cargan primero en un esquema de staging (stg) y de allí se transforman al esquema dimensional (dwh).

Tabla	Tipo	Clave primaria	Claves foráneas	Descripción
fact_ventas	Hechos	venta_key	tiempo_key, producto_key, cliente_key	Un ítem vendido dentro de un pedido; contiene las medidas
dim_tiempo	Dimensión	tiempo_key (YYYYMMDD)	—	Calendario 2016–2018: año, trimestre, mes, nombre del mes, día de la semana
dim_producto	Dimensión	producto_key	—	Producto con su categoría (portugués e inglés) y peso
dim_cliente	Dimensión	cliente_key	—	Cliente real (customer_unique_id) con ciudad y estado

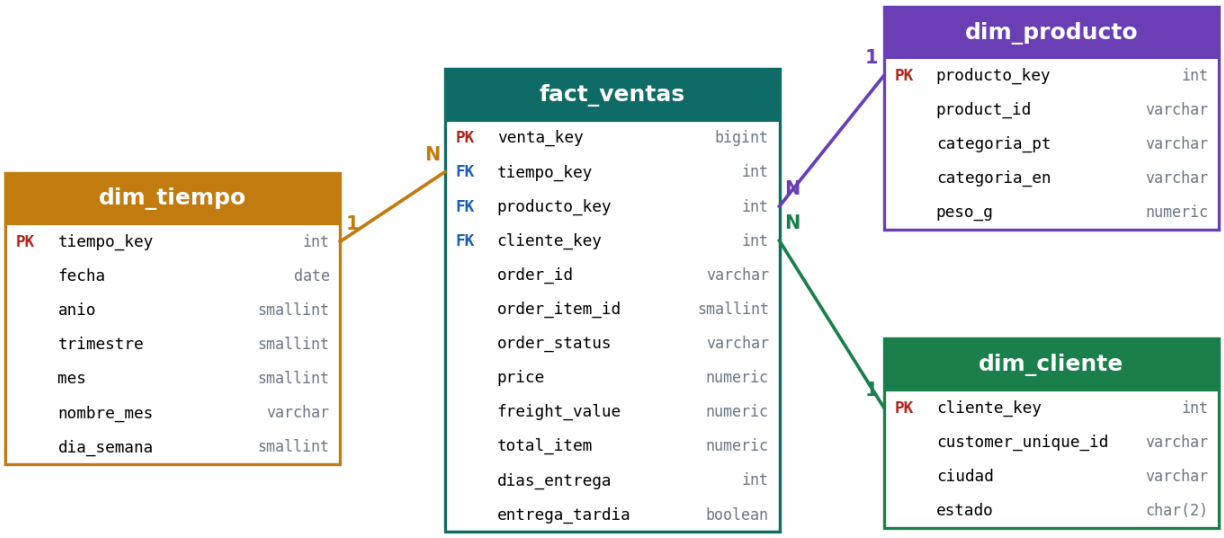
Medidas de la tabla de hechos: price (precio), freight_value (flete), total_item (precio + flete), dias_entrega y entrega_tardia. Con ellas se calculan los tres KPI: ventas totales, total de pedidos y ticket promedio.

Preparación y limpieza de datos

Transformaciones aplicadas al dataset original:

Problema detectado	Tratamiento
610 productos sin categoría	Se asigna categoria_en = 'sin_categoria'
Categorías en portugués	Se traducen con product_category_name_translation.csv; si no hay traducción se conserva el nombre original
customer_id (99.441) distinto de clientes reales (96.096)	dim_cliente usa customer_unique_id, un registro por cliente real
Fechas de entrega nulas (pedidos no entregados)	dias_entrega y entrega_tardia quedan en NULL; no se inventan valores
Pedidos canceled y unavailable	Se conservan en hechos; las funciones SQL los excluyen del cálculo de ventas
Pagos, reseñas, vendedores y geolocalización	Fuera del alcance del modelo mínimo. Un pedido tiene varios pagos y unirlos al ítem duplicaría los importes

2. Diagrama estrella



Esquema estrella · granularidad: un ítem vendido dentro de un pedido

Figura 1. Esquema estrella del Data Mart (PK en rojo, FK en azul). Cada fila de fact_ventas referencia una fila de cada dimensión (relación N a 1).

3. Descripción de granularidad

Una fila de la tabla fact_ventas representa un ítem (producto) vendido dentro de un pedido.

La clave natural de la granularidad es order_id + order_item_id, protegida con la restricción UNIQUE uq_fact_pedido_item. Se eligió este nivel, el más detallado disponible, porque permite agregar hacia arriba (por pedido, mes, categoría o estado) sin perder información.

Consecuencia a tener en cuenta: un pedido con varios productos genera varias filas. Por eso el KPI de pedidos se calcula con COUNT(DISTINCT order_id) y no con COUNT(*).

Los datos confirman la granularidad: la tabla de hechos tiene 112.650 filas, igual que olist_order_items, y 0 duplicados de (order_id, order_item_id).

4. Scripts de creación de tablas

```
-- ================================================================
-- 01 · Esquemas, STAGING (CSV tal cual) y MODELO DIMENSIONAL
-- Granularidad de fact_ventas: UNA FILA = UN ÍTEM (producto) VENDIDO
-- DENTRO DE UN PEDIDO (order_id + order_item_id).
-- ================================================================
CREATE SCHEMA stg;
CREATE SCHEMA dwh;

-- ---------- STAGING: todo en TEXT, mismo orden que los CSV ----------
CREATE TABLE stg.customers (
   customer_id TEXT, customer_unique_id TEXT, customer_zip_code_prefix TEXT,
   customer_city TEXT, customer_state TEXT);

CREATE TABLE stg.orders (
   order_id TEXT, customer_id TEXT, order_status TEXT,
   order_purchase_timestamp TEXT, order_approved_at TEXT,
   order_delivered_carrier_date TEXT, order_delivered_customer_date TEXT,
   order_estimated_delivery_date TEXT);

CREATE TABLE stg.order_items (
   order_id TEXT, order_item_id TEXT, product_id TEXT, seller_id TEXT,
   shipping_limit_date TEXT, price TEXT, freight_value TEXT);

CREATE TABLE stg.products (
   product_id TEXT, product_category_name TEXT, product_name_lenght TEXT,
   product_description_lenght TEXT, product_photos_qty TEXT, product_weight_g TEXT,
   product_length_cm TEXT, product_height_cm TEXT, product_width_cm TEXT);

CREATE TABLE stg.category_translation (
   product_category_name TEXT, product_category_name_english TEXT);

-- ---------- DIMENSIONES ----------
CREATE TABLE dwh.dim_tiempo (
   tiempo_key  INT        PRIMARY KEY,            -- YYYYMMDD
   fecha       DATE       NOT NULL UNIQUE,
   anio        SMALLINT   NOT NULL CHECK (anio BETWEEN 2000 AND 2100),
   trimestre   SMALLINT   NOT NULL CHECK (trimestre BETWEEN 1 AND 4),
   mes         SMALLINT   NOT NULL CHECK (mes BETWEEN 1 AND 12),
   nombre_mes  VARCHAR(15) NOT NULL,
   dia_semana  SMALLINT   NOT NULL CHECK (dia_semana BETWEEN 1 AND 7)  -- 1 = lunes
);

CREATE TABLE dwh.dim_producto (
   producto_key  SERIAL       PRIMARY KEY,
   product_id    VARCHAR(40)  NOT NULL UNIQUE,
   categoria_pt  VARCHAR(80),
   categoria_en  VARCHAR(80)  NOT NULL DEFAULT 'sin_categoria',
   peso_g        NUMERIC(10,2) CHECK (peso_g IS NULL OR peso_g >= 0)
);

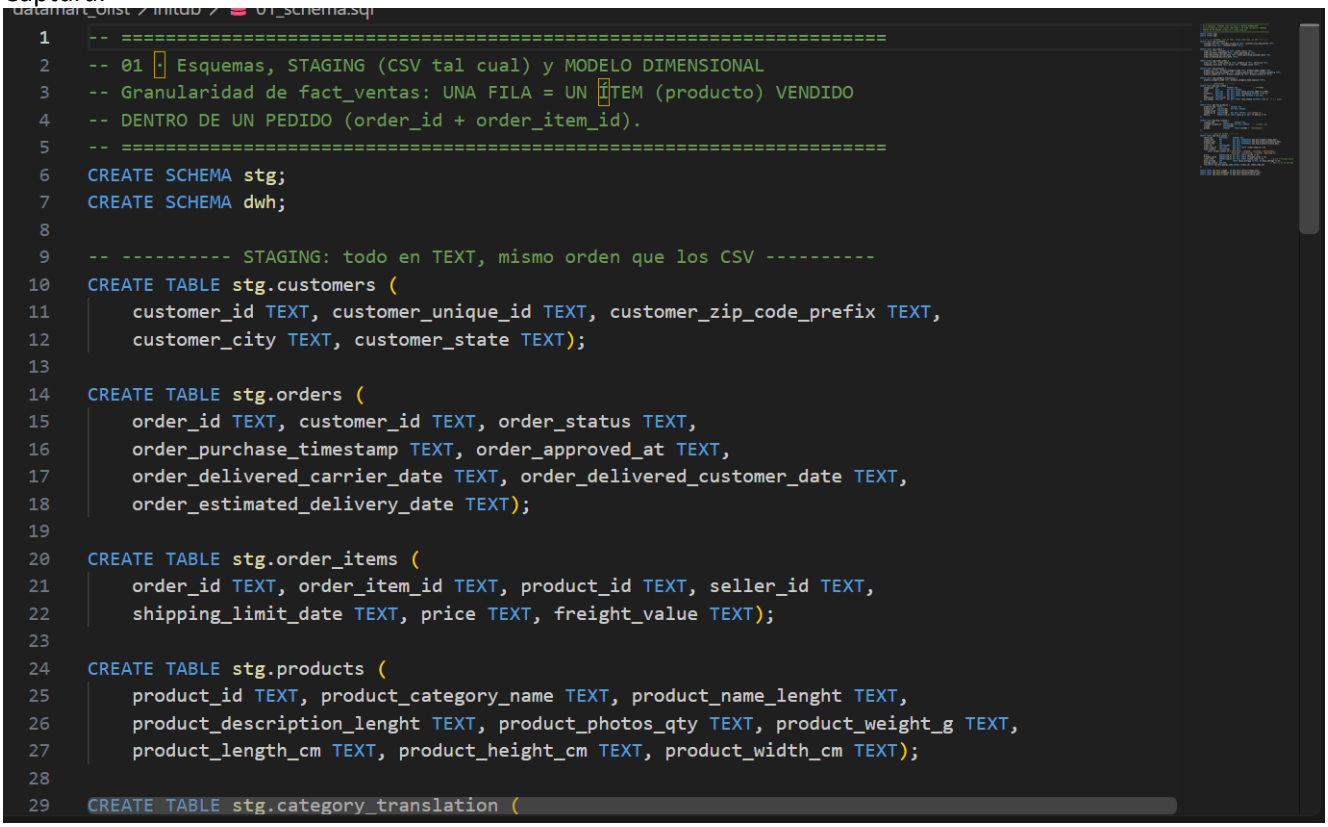
CREATE TABLE dwh.dim_cliente (
   cliente_key        SERIAL     PRIMARY KEY,
```

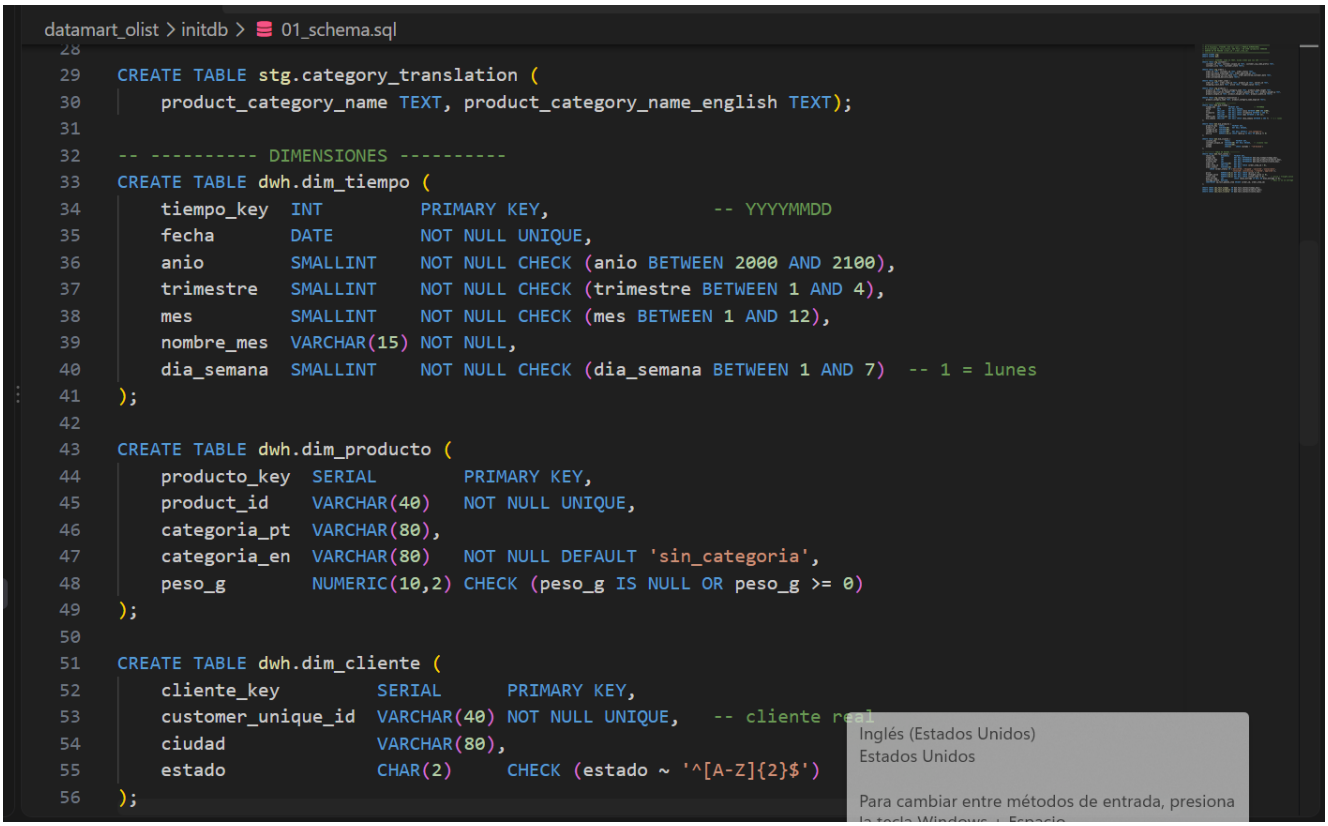
```
    customer_unique_id  VARCHAR(40) NOT NULL UNIQUE,   -- cliente real
    ciudad              VARCHAR(80),
    estado              CHAR(2)     CHECK (estado ~ '^[A-Z]{2}$')
);

-- ---------- TABLA DE HECHOS ----------
CREATE TABLE dwh.fact_ventas (
    venta_key       BIGSERIAL     PRIMARY KEY,
    tiempo_key      INT           NOT NULL REFERENCES dwh.dim_tiempo(tiempo_key),
    producto_key    INT           NOT NULL REFERENCES dwh.dim_producto(producto_key),
    cliente_key     INT           NOT NULL REFERENCES dwh.dim_cliente(cliente_key),
    order_id        VARCHAR(40)   NOT NULL,
    order_item_id   SMALLINT      NOT NULL CHECK (order_item_id > 0),
    order_status    VARCHAR(20)   NOT NULL
        CHECK (order_status IN ('delivered','shipped','canceled','unavailable',
                                'invoiced','processing','created','approved')),
    price           NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    freight_value   NUMERIC(10,2) NOT NULL CHECK (freight_value >= 0),
    total_item      NUMERIC(10,2) NOT NULL CHECK (total_item >= 0),      -- price + freight_value
    dias_entrega    INT           CHECK (dias_entrega IS NULL OR dias_entrega >= 0),
    entrega_tardia  BOOLEAN,                                           -- NULL si no se entregó
    CONSTRAINT uq_fact_pedido_item UNIQUE (order_id, order_item_id)
);

CREATE INDEX idx_fact_tiempo   ON dwh.fact_ventas(tiempo_key);
CREATE INDEX idx_fact_producto ON dwh.fact_ventas(producto_key);
CREATE INDEX idx_fact_cliente  ON dwh.fact_ventas(cliente_key);
```

Captura:





```
datamart_olist > initdb > 01_schema.sql

CREATE TABLE stg.category_translation (
    product_category_name TEXT, product_category_name_english TEXT);

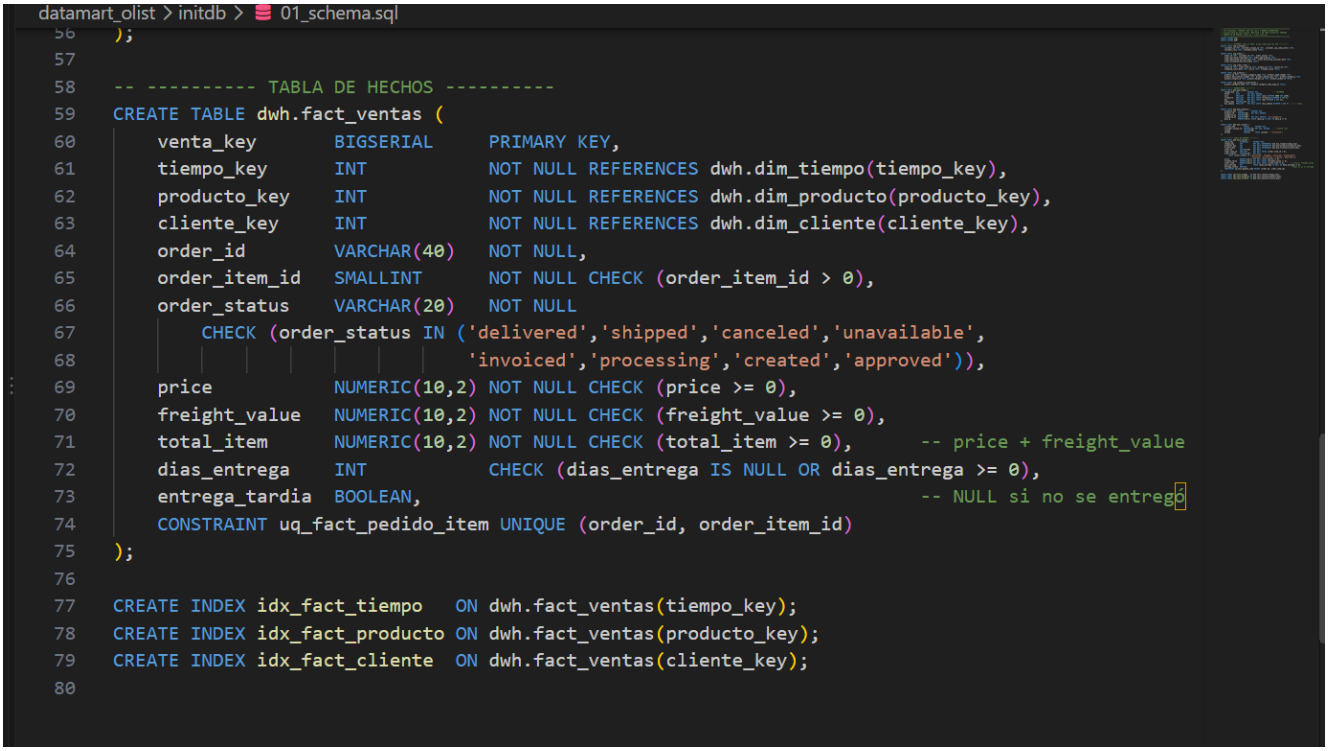
-- ---------- DIMENSIONES ----------
CREATE TABLE dwh.dim_tiempo (
    tiempo_key  INT         PRIMARY KEY,              -- YYYYMMDD
    fecha       DATE        NOT NULL UNIQUE,
    anio        SMALLINT    NOT NULL CHECK (anio BETWEEN 2000 AND 2100),
    trimestre   SMALLINT    NOT NULL CHECK (trimestre BETWEEN 1 AND 4),
    mes         SMALLINT    NOT NULL CHECK (mes BETWEEN 1 AND 12),
    nombre_mes  VARCHAR(15) NOT NULL,
    dia_semana  SMALLINT    NOT NULL CHECK (dia_semana BETWEEN 1 AND 7)  -- 1 = lunes
);

CREATE TABLE dwh.dim_producto (
    producto_key  SERIAL        PRIMARY KEY,
    product_id    VARCHAR(40)   NOT NULL UNIQUE,
    categoria_pt  VARCHAR(80),
    categoria_en  VARCHAR(80)   NOT NULL DEFAULT 'sin_categoria',
    peso_g        NUMERIC(10,2) CHECK (peso_g IS NULL OR peso_g >= 0)
);

CREATE TABLE dwh.dim_cliente (
    cliente_key         SERIAL      PRIMARY KEY,
    customer_unique_id  VARCHAR(40) NOT NULL UNIQUE,   -- cliente real
    ciudad              VARCHAR(80),
    estado              CHAR(2)     CHECK (estado ~ '^[A-Z]{2}$')
);
```

Inglés (Estados Unidos)
Estados Unidos

Para cambiar entre métodos de entrada, presiona



```
datamart_olist > initdb > 01_schema.sql

);

-- ---------- TABLA DE HECHOS ----------
CREATE TABLE dwh.fact_ventas (
    venta_key       BIGSERIAL     PRIMARY KEY,
    tiempo_key      INT           NOT NULL REFERENCES dwh.dim_tiempo(tiempo_key),
    producto_key    INT           NOT NULL REFERENCES dwh.dim_producto(producto_key),
    cliente_key     INT           NOT NULL REFERENCES dwh.dim_cliente(cliente_key),
    order_id        VARCHAR(40)   NOT NULL,
    order_item_id   SMALLINT      NOT NULL CHECK (order_item_id > 0),
    order_status    VARCHAR(20)   NOT NULL
        CHECK (order_status IN ('delivered','shipped','canceled','unavailable',
                                'invoiced','processing','created','approved')),
    price           NUMERIC(10,2) NOT NULL CHECK (price >= 0),
    freight_value   NUMERIC(10,2) NOT NULL CHECK (freight_value >= 0),
    total_item      NUMERIC(10,2) NOT NULL CHECK (total_item >= 0),      -- price + freight_value
    dias_entrega    INT           CHECK (dias_entrega IS NULL OR dias_entrega >= 0),
    entrega_tardia  BOOLEAN,                                           -- NULL si no se entregó
    CONSTRAINT uq_fact_pedido_item UNIQUE (order_id, order_item_id)
);

CREATE INDEX idx_fact_tiempo   ON dwh.fact_ventas(tiempo_key);
CREATE INDEX idx_fact_producto ON dwh.fact_ventas(producto_key);
CREATE INDEX idx_fact_cliente  ON dwh.fact_ventas(cliente_key);
```

5. Evidencia de carga de datos

La carga se ejecuta automáticamente al levantar Docker con el archivo initdb/02_carga.sql, en cuatro pasos: (1) COPY de los CSV a staging, (2) generación de dim_tiempo con generate_series, (3) carga de dim_cliente y dim_producto con las transformaciones de limpieza, y (4) carga de fact_ventas uniendo ítems, pedidos, clientes y productos.

Tabla	Filas cargadas	Origen
dwh.fact_ventas	112.650	olist_order_items + olist_orders
dwh.dim_cliente	96.096	olist_customers (customer_unique_id distintos)
dwh.dim_producto	32.951	olist_products + traducción de categorías
dwh.dim_tiempo	1.096	Generada (2016-01-01 a 2018-12-31)

Salida de la ejecución de validación

```
=== 1. CANTIDAD DE REGISTROS (staging vs. modelo) ===
                    tabla                     | filas
----------------------------------------------+--------
 dwh.dim_tiempo                               |   1096
 stg.products                                 |  32951
 dwh.dim_producto                             |  32951
 dwh.dim_cliente                              |  96096
 dwh.fact_ventas                              | 112650
 stg.order_items                              | 112650
 stg.customers (customer_unique_id distintos) |  96096
(7 rows)


=== 2. INTEGRIDAD: hechos sin dimensión (debe dar 0 en todas) ===
 huerfanos_tiempo | huerfanos_producto | huerfanos_cliente
------------------+--------------------+-------------------
                0 |                  0 |                 0
(1 row)


=== 3. INTEGRIDAD: granularidad sin duplicados (debe dar 0) ===
 duplicados_pedido_item
------------------------
                      0
(1 row)


=== 4. VALORES NULOS RELEVANTES ===
 sin_fecha_entrega | sin_dato_entrega_tardia | sin_precio_o_flete
-------------------+-------------------------+--------------------
              2454 |                    2454 |                  0
(1 row)


 productos_sin_categoria | productos_sin_peso
-------------------------+--------------------
                     610 |                  2
(1 row)


=== 5. TOTALES: staging vs. hechos (deben coincidir) ===
 ventas_staging | ventas_dwh  | flete_staging | flete_dwh
----------------+-------------+---------------+------------
    13591643.70 | 13591643.70 |    2251909.54 | 2251909.54
(1 row)


=== 6. KPIs PRINCIPALES (sin filtros) ===
 ventas_totales | total_pedidos | ticket_promedio
----------------+---------------+-----------------
    13494400.74 |         98199 |          137.42
(1 row)


=== 7. VENTAS POR MES (año 2017) ===
 periodo_anio | periodo_mes | mes_nombre | ventas_totales | total_pedidos | ticket_promedio
--------------+-------------+------------+----------------+---------------+-----------------
         2017 |           1 | Enero      |      120098.27 |           787 |          152.60
         2017 |           2 | Febrero    |      244959.35 |          1718 |          142.58
         2017 |           3 | Marzo      |      368341.32 |          2617 |          140.75
         2017 |           4 | Abril      |      353842.98 |          2377 |          148.86
         2017 |           5 | Mayo       |      503159.19 |          3640 |          138.23
         2017 |           6 | Junio      |      429916.61 |          3205 |          134.14
         2017 |           7 | Julio      |      492287.30 |          3946 |          124.76
         2017 |           8 | Agosto     |      568245.79 |          4272 |          133.02
         2017 |           9 | Septiembre |      621415.91 |          4227 |          147.01
         2017 |          10 | Octubre    |      660179.62 |          4547 |          145.19
         2017 |          11 | Noviembre  |     1003862.14 |          7421 |          135.27
```

```
        2017 |          12 | Diciembre  |      742183.79 |          5618 |          132.11
(12 rows)

=== 8. TOP 5 CATEGORÍAS ===
     categoria_en      | ventas_totales | total_pedidos
-----------------------+----------------+---------------
 health_beauty         |     1255695.13 |          8800
 watches_gifts         |     1198185.21 |          5604
 bed_bath_table        |     1035964.06 |          9399
 sports_leisure        |      979740.92 |          7673
 computers_accessories |      904322.02 |          6654
(5 rows)
```

```
PS C:\Users\jgonz\Downloads\datamart_olist (1)\datamart_olist> docker compose exec db psql -U postgres -d olist -f
/validacion/validacion.sql
=== 1. CANTIDAD DE REGISTROS (staging vs. modelo) ===
                  tabla                   | filas
------------------------------------------+--------
 dwh.dim_tiempo                           |   1096
 stg.products                             |  32951
 dwh.dim_producto                         |  32951
 dwh.dim_cliente                          |  96096
 stg.order_items                          | 112650
 dwh.fact_ventas                          | 112650
 stg.customers (customer_unique_id distintos) |  96096
(7 rows)

=== 2. INTEGRIDAD: hechos sin dimensión (debe dar 0 en todas) ===
 huerfanos_tiempo | huerfanos_producto | huerfanos_cliente
------------------+--------------------+-------------------
                0 |                  0 |                 0
(1 row)

=== 3. INTEGRIDAD: granularidad sin duplicados (debe dar 0) ===
 duplicados_pedido_item
------------------------
                      0
                      0
(1 row)

=== 4. VALORES NULOS RELEVANTES ===
 sin_fecha_entrega | sin_dato_entrega_tardia | sin_precio_o_flete
-------------------+-------------------------+--------------------
              2454 |                    2454 |                  0
(1 row)

 productos_sin_categoria | productos_sin_peso
-------------------------+--------------------
                     610 |                  2
(1 row)

=== 5. TOTALES: staging vs. hechos (deben coincidir) ===
 ventas_staging | ventas_dwh  | flete_staging | flete_dwh
----------------+-------------+---------------+------------
    13591643.70 | 13591643.70 |    2251909.54 | 2251909.54
(1 row)

=== 6. KPIs PRINCIPALES (sin filtros) ===
 ventas_totales | total_pedidos | ticket_promedio
```

```
/validacion/validacion.sql
---------------+--------------+----------------
    13494400.74 |        98199 |         137.42
(1 row)

=== 7. VENTAS POR MES (año 2017) ===
 periodo_anio | periodo_mes | mes_nombre | ventas_totales | total_pedidos | ticket_promedio
--------------+-------------+------------+----------------+---------------+-----------------
         2017 |           1 | Enero      |      120098.27 |           787 |          152.60
         2017 |           2 | Febrero    |      244959.35 |          1718 |          142.58
         2017 |           3 | Marzo      |      368341.32 |          2617 |          140.75
         2017 |           4 | Abril      |      353842.98 |          2377 |          148.86
         2017 |           5 | Mayo       |      503159.19 |          3640 |          138.23
         2017 |           6 | Junio      |      429916.61 |          3205 |          134.14
         2017 |           7 | Julio      |      492287.30 |          3946 |          124.76
         2017 |           8 | Agosto     |      568245.79 |          4272 |          133.02
         2017 |           9 | Septiembre |      621415.91 |          4227 |          147.01
         2017 |          10 | Octubre    |      660179.62 |          4547 |          145.19
         2017 |          11 | Noviembre  |     1003862.14 |          7421 |          135.27
         2017 |          12 | Diciembre  |      742183.79 |          5618 |          132.11
(12 rows)

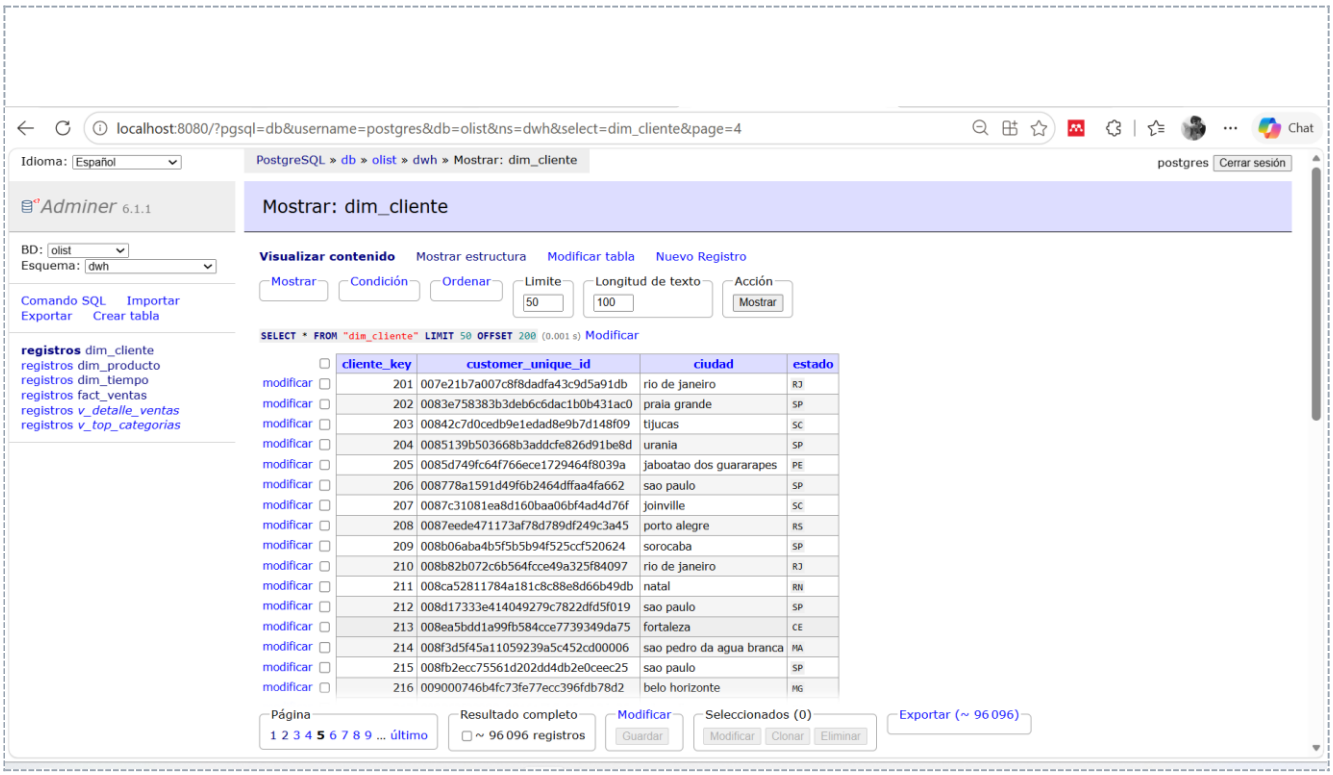
=== 8. TOP 5 CATEGORÍAS ===
```

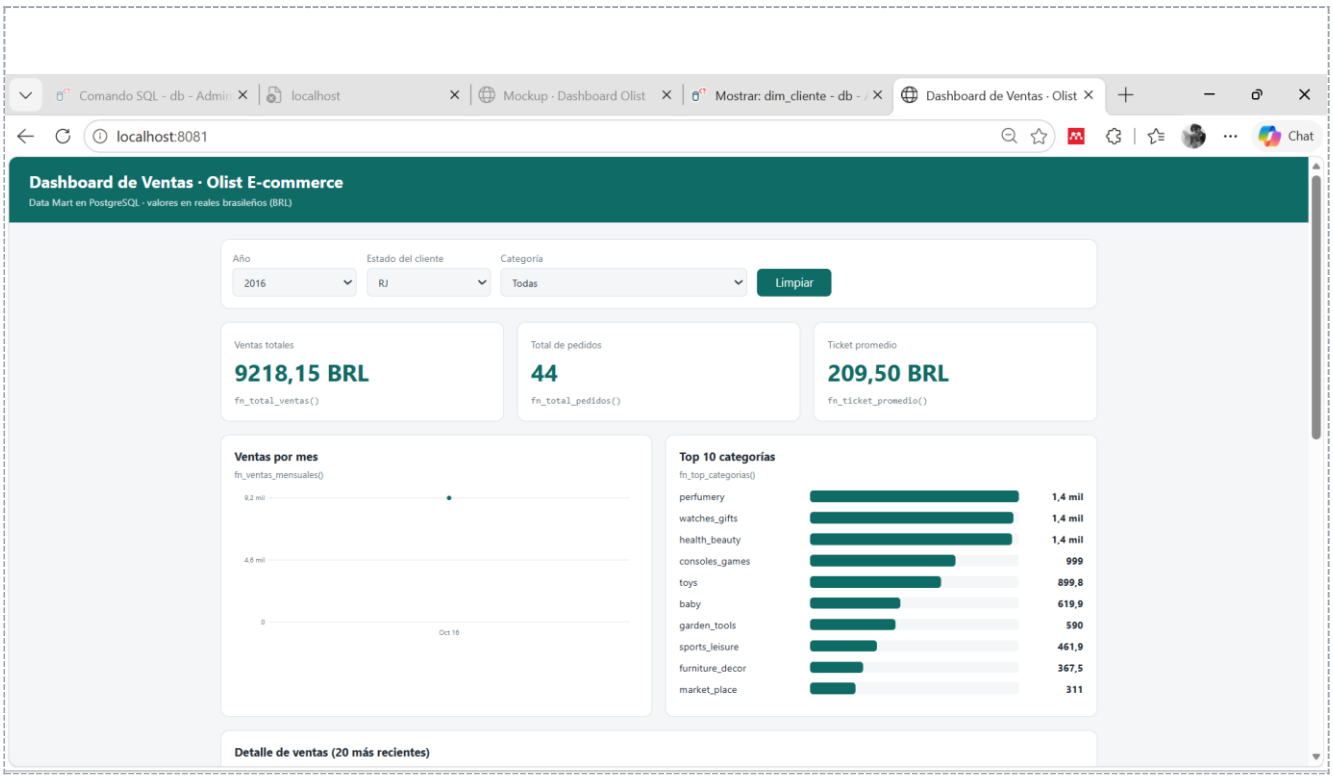
```
=== 8. TOP 5 CATEGORÍAS ===
     categoria_en      | ventas_totales | total_pedidos
-----------------------+----------------+---------------
 health_beauty         |     1255695.13 |          8800
 watches_gifts         |     1198185.21 |          5604
 bed_bath_table        |     1035964.06 |          9399
 sports_leisure        |      979740.92 |          7673
 computers_accessories |      904322.02 |          6654
(5 rows)

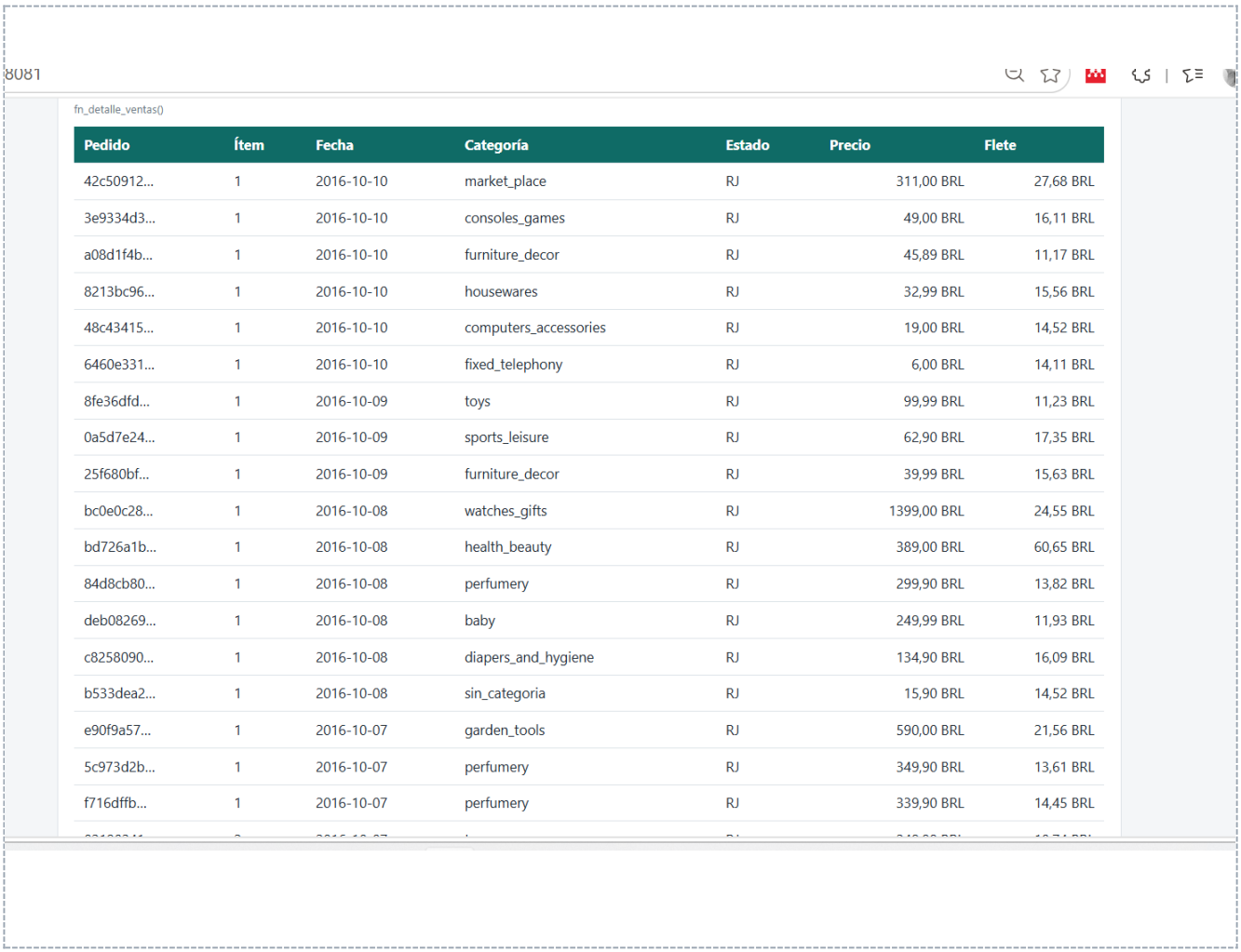
PS C:\Users\jgonz\Downloads\datamart_olist (1)\datamart_olist>
```

Capturas de la ejecución

[Insertar captura: docker compose ps con los tres contenedores en estado healthy]


localhost:8080/?pgsql=db&username=postgres&db=olist&ns=dwh&select=dim_cliente&page=4
Idioma: Español
PostgreSQL » db » olist » dwh » Mostrar: dim_cliente
postgres Cerrar sesión
Adminer 6.1.1
BD: olist
Esquema: dwh
Comando SQL Importar
Exportar Crear tabla
registros dim_cliente
registros dim_producto
registros dim_tiempo
registros fact_ventas
registros v_detalle_ventas
registros v_top_categorias
Mostrar: dim_cliente
Visualizar contenido Mostrar estructura Modificar tabla Nuevo Registro
Mostrar Condición Ordenar Limite Longitud de texto Acción
50 100 Mostrar
SELECT * FROM "dim_cliente" LIMIT 50 OFFSET 200 (0.001 s) Modificar
cliente_key customer_unique_id ciudad estado
modificar 201 007e21b7a007c8f8dadfa43c9d5a91db rio de janeiro RJ
modificar 202 0083e758383b3deb6c6dac1b0b431ac0 praia grande SP
modificar 203 00842c7d0cedb9e1edad8e9b7d148f09 tijucas SC
modificar 204 0085139b503668b3addcfe826d91be8d urania SP
modificar 205 0085d749fc64f766ece1729464f8039a jaboatao dos guararapes PE
modificar 206 008778a1591d49f6b2464dffaa4fa662 sao paulo SP
modificar 207 0087c31081ea8d160baa06bf4ad4d76f joinville SC
modificar 208 0087eede471173af78d789df249c3a45 porto alegre RS
modificar 209 008b06aba4b5f5b5b94f525ccf520624 sorocaba SP
modificar 210 008b82b072c6b564fcce49a325f84097 rio de janeiro RJ
modificar 211 008ca52811784a181c8c88e8d66b49db natal RN
modificar 212 008d17333e414049279c7822dfd5f019 sao paulo SP
modificar 213 008ea5bdd1a99fb584cce7739349da75 fortaleza CE
modificar 214 008f3d5f45a11059239a5c452cd00006 sao pedro da agua branca MA
modificar 215 008fb2ecc75561d202dd4db2e0ceec25 sao paulo SP
modificar 216 009000746b4fc73fe77ecc396fdb78d2 belo horizonte MG
Página 1 2 3 4 5 6 7 8 9 ... último
Resultado completo ~ 96 096 registros
Modificar Guardar
Seleccionados (0) Modificar Clonar Eliminar
Exportar (~ 96 096)


Comando SQL - db - Admir
localhost
Mockup · Dashboard Olist
Mostrar: dim_cliente - db -
Dashboard de Ventas · Olist
localhost:8081
Dashboard de Ventas · Olist E-commerce
Data Mart en PostgreSQL · valores en reales brasileños (BRL)
Año 2016
Estado del cliente RJ
Categoría Todas
Limpiar
Ventas totales
9218,15 BRL
fn_total_ventas()
Total de pedidos
44
fn_total_pedidos()
Ticket promedio
209,50 BRL
fn_ticket_promedio()
Ventas por mes
fn_ventas_mensuales()
9,2 mil
4,6 mil
0
Oct 16
Top 10 categorías
fn_top_categorias()
perfumery 1,4 mil
watches_gifts 1,4 mil
health_beauty 1,4 mil
consoles_games 999
toys 899,8
baby 619,9
garden_tools 590
sports_leisure 461,9
furniture_decor 367,5
market_place 311
Detalle de ventas (20 más recientes)



fn_detalle_ventas()

Pedido	Ítem	Fecha	Categoría	Estado	Precio	Flete
42c50912...	1	2016-10-10	market_place	RJ	311,00 BRL	27,68 BRL
3e9334d3...	1	2016-10-10	consoles_games	RJ	49,00 BRL	16,11 BRL
a08d1f4b...	1	2016-10-10	furniture_decor	RJ	45,89 BRL	11,17 BRL
8213bc96...	1	2016-10-10	housewares	RJ	32,99 BRL	15,56 BRL
48c43415...	1	2016-10-10	computers_accessories	RJ	19,00 BRL	14,52 BRL
6460e331...	1	2016-10-10	fixed_telephony	RJ	6,00 BRL	14,11 BRL
8fe36dfd...	1	2016-10-09	toys	RJ	99,99 BRL	11,23 BRL
0a5d7e24...	1	2016-10-09	sports_leisure	RJ	62,90 BRL	17,35 BRL
25f680bf...	1	2016-10-09	furniture_decor	RJ	39,99 BRL	15,63 BRL
bc0e0c28...	1	2016-10-08	watches_gifts	RJ	1399,00 BRL	24,55 BRL
bd726a1b...	1	2016-10-08	health_beauty	RJ	389,00 BRL	60,65 BRL
84d8cb80...	1	2016-10-08	perfumery	RJ	299,90 BRL	13,82 BRL
deb08269...	1	2016-10-08	baby	RJ	249,99 BRL	11,93 BRL
c8258090...	1	2016-10-08	diapers_and_hygiene	RJ	134,90 BRL	16,09 BRL
b533dea2...	1	2016-10-08	sin_categoria	RJ	15,90 BRL	14,52 BRL
e90f9a57...	1	2016-10-07	garden_tools	RJ	590,00 BRL	21,56 BRL
5c973d2b...	1	2016-10-07	perfumery	RJ	349,90 BRL	13,61 BRL
f716dffb...	1	2016-10-07	perfumery	RJ	339,90 BRL	14,45 BRL

Script de carga

```
-- ============================================================================
-- 02 · CARGA: CSV -> staging (COPY) -> dimensiones -> hechos
-- ============================================================================


-- ---------- 1) CSV a staging ----------
COPY stg.customers            FROM '/data/olist_customers_dataset.csv'      WITH (FORMAT csv, HEADER true);
COPY stg.orders               FROM '/data/olist_orders_dataset.csv'         WITH (FORMAT csv, HEADER true);
COPY stg.order_items          FROM '/data/olist_order_items_dataset.csv'    WITH (FORMAT csv, HEADER true);
COPY stg.products             FROM '/data/olist_products_dataset.csv'       WITH (FORMAT csv, HEADER true);
COPY stg.category_translation FROM '/data/product_category_name_translation.csv' WITH (FORMAT csv, HEADER true);


-- ---------- 2) dim_tiempo (generada) ----------
INSERT INTO dwh.dim_tiempo
SELECT TO_CHAR(d, 'YYYYMMDD')::INT,
       d::DATE,
       EXTRACT(YEAR    FROM d)::SMALLINT,
       EXTRACT(QUARTER FROM d)::SMALLINT,
       EXTRACT(MONTH   FROM d)::SMALLINT,
       (ARRAY['Enero','Febrero','Marzo','Abril','Mayo','Junio','Julio',
              'Agosto','Septiembre','Octubre','Noviembre','Diciembre'])[EXTRACT(MONTH FROM d)::INT],
       EXTRACT(ISODOW  FROM d)::SMALLINT
FROM generate_series('2016-01-01'::DATE, '2018-12-31'::DATE, INTERVAL '1 day') AS d;


-- ---------- 3) dim_cliente: un registro por cliente REAL (customer_unique_id) ----------
INSERT INTO dwh.dim_cliente (customer_unique_id, ciudad, estado)
SELECT DISTINCT ON (customer_unique_id)
       customer_unique_id, customer_city, UPPER(customer_state)
FROM stg.customers
ORDER BY customer_unique_id, customer_id;


-- ---------- 4) dim_producto: traduce categoría; nulos -> 'sin_categoria' ----------
INSERT INTO dwh.dim_producto (product_id, categoria_pt, categoria_en, peso_g)
SELECT p.product_id,
       p.product_category_name,
       COALESCE(t.product_category_name_english,   -- traducción
                p.product_category_name,           -- sin traducción: deja el nombre original
                'sin_categoria'),                  -- categoría nula
       p.product_weight_g::NUMERIC
FROM stg.products p
```

```
LEFT JOIN stg.category_translation t USING (product_category_name);


-- ---------- 5) fact_ventas: 1 fila por ítem de pedido ----------
INSERT INTO dwh.fact_ventas
    (tiempo_key, producto_key, cliente_key, order_id, order_item_id, order_status,
     price, freight_value, total_item, dias_entrega, entrega_tardia)
SELECT TO_CHAR(o.order_purchase_timestamp::TIMESTAMP, 'YYYYMMDD')::INT,
       p.producto_key,
       c.cliente_key,
       i.order_id,
       i.order_item_id::SMALLINT,
       o.order_status,
       i.price::NUMERIC,
       i.freight_value::NUMERIC,
       i.price::NUMERIC + i.freight_value::NUMERIC,
       CASE WHEN o.order_delivered_customer_date IS NOT NULL
            THEN o.order_delivered_customer_date::DATE - o.order_purchase_timestamp::DATE END,
       CASE WHEN o.order_delivered_customer_date IS NOT NULL
            THEN o.order_delivered_customer_date::TIMESTAMP > o.order_estimated_delivery_date::TIMESTAMP END
FROM stg.order_items i
JOIN stg.orders      o  ON o.order_id    = i.order_id
JOIN stg.customers   sc ON sc.customer_id = o.customer_id
JOIN dwh.dim_cliente c  ON c.customer_unique_id = sc.customer_unique_id
JOIN dwh.dim_producto p ON p.product_id  = i.product_id;

ANALYZE;
```

6. Consultas de validación

Archivo validacion/validacion.sql. Cubre los cuatro aspectos solicitados: cantidad de registros, integridad de relaciones, valores nulos relevantes y totales de las medidas principales.

Validación	Qué comprueba	Resultado obtenido
1. Cantidad de registros	Staging y modelo tienen las mismas filas	112.650 en hechos = 112.650 en staging; 96.096 clientes; 32.951 productos; 1.096 días
2. Integridad de relaciones	Hechos sin dimensión asociada	0 huérfanos en tiempo, producto y cliente
3. Granularidad	Duplicados de (order_id, order_item_id)	0 duplicados
4. Valores nulos relevantes	Entregas sin fecha; productos sin categoría o peso	2.454 ítems sin fecha de entrega (no entregados); 610 productos sin categoría; 2 sin peso
5. Totales	Ventas y flete: staging frente a modelo	Ventas 13.591.643,70 y flete 2.251.909,54, idénticos en ambos
6. KPI principales	Ventas, pedidos y ticket sin filtros (excluye cancelados)	R$ 13.494.400,74 · 98.199 pedidos · ticket R$ 137,42
7. Ventas por mes	Función mensual con filtro de año	12 meses de 2017 con ventas, pedidos y ticket
8. Top categorías	Vista de categorías	health_beauty, watches_gifts, bed_bath_table, sports_leisure, computers_accessories

Código

```
-- ================================================================
-- CONSULTAS DE VALIDACIÓN
-- Ejecutar:  docker compose exec db psql -U postgres -d olist -f /validacion/validacion.sql
-- ================================================================
\echo '=== 1. CANTIDAD DE REGISTROS (staging vs. modelo) ==='
SELECT 'stg.order_items' AS tabla, COUNT(*) AS filas FROM stg.order_items
UNION ALL SELECT 'dwh.fact_ventas',  COUNT(*) FROM dwh.fact_ventas
UNION ALL SELECT 'stg.customers (customer_unique_id distintos)', COUNT(DISTINCT customer_unique_id)
FROM stg.customers
UNION ALL SELECT 'dwh.dim_cliente',  COUNT(*) FROM dwh.dim_cliente
```

```
UNION ALL SELECT 'stg.products',     COUNT(*) FROM stg.products
UNION ALL SELECT 'dwh.dim_producto', COUNT(*) FROM dwh.dim_producto
UNION ALL SELECT 'dwh.dim_tiempo',   COUNT(*) FROM dwh.dim_tiempo;

\echo '=== 2. INTEGRIDAD: hechos sin dimensión (debe dar 0 en todas) ==='
SELECT
  (SELECT COUNT(*) FROM dwh.fact_ventas f LEFT JOIN dwh.dim_tiempo   d ON d.tiempo_key   = f.tiempo_key
WHERE d.tiempo_key   IS NULL) AS huerfanos_tiempo,
  (SELECT COUNT(*) FROM dwh.fact_ventas f LEFT JOIN dwh.dim_producto d ON d.producto_key =
f.producto_key WHERE d.producto_key IS NULL) AS huerfanos_producto,
  (SELECT COUNT(*) FROM dwh.fact_ventas f LEFT JOIN dwh.dim_cliente  d ON d.cliente_key  = f.cliente_key
WHERE d.cliente_key  IS NULL) AS huerfanos_cliente;

\echo '=== 3. INTEGRIDAD: granularidad sin duplicados (debe dar 0) ==='
SELECT COUNT(*) AS duplicados_pedido_item FROM (
  SELECT order_id, order_item_id FROM dwh.fact_ventas GROUP BY 1,2 HAVING COUNT(*) > 1) x;

\echo '=== 4. VALORES NULOS RELEVANTES ==='
SELECT
  COUNT(*) FILTER (WHERE dias_entrega   IS NULL) AS sin_fecha_entrega,
  COUNT(*) FILTER (WHERE entrega_tardia IS NULL) AS sin_dato_entrega_tardia,
  COUNT(*) FILTER (WHERE price IS NULL OR freight_value IS NULL) AS sin_precio_o_flete
FROM dwh.fact_ventas;
SELECT COUNT(*) FILTER (WHERE categoria_pt IS NULL) AS productos_sin_categoria,
     COUNT(*) FILTER (WHERE peso_g IS NULL)       AS productos_sin_peso
FROM dwh.dim_producto;

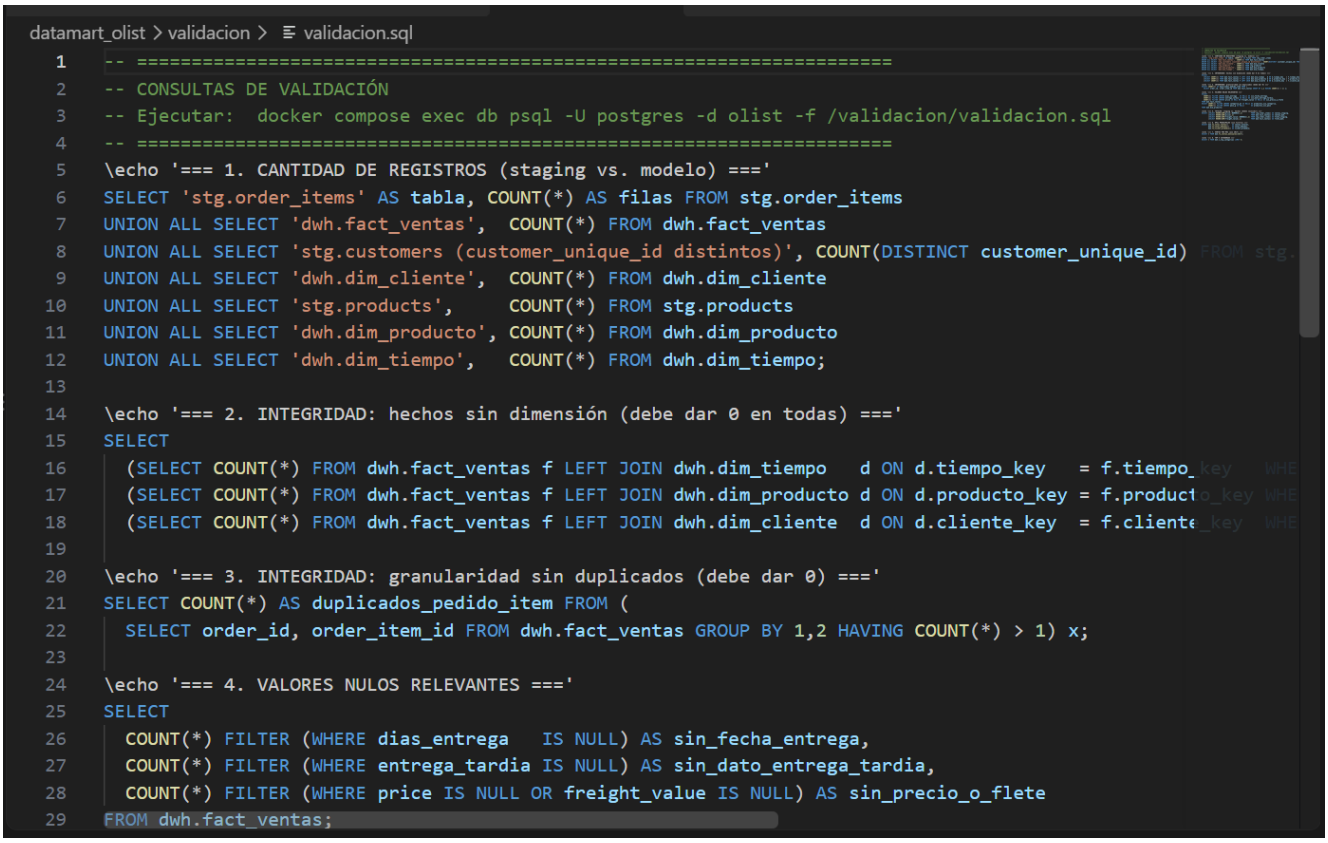
\echo '=== 5. TOTALES: staging vs. hechos (deben coincidir) ==='
SELECT (SELECT ROUND(SUM(price::NUMERIC),2)        FROM stg.order_items) AS ventas_staging,
     (SELECT ROUND(SUM(price),2)                FROM dwh.fact_ventas) AS ventas_dwh,
     (SELECT ROUND(SUM(freight_value::NUMERIC),2) FROM stg.order_items) AS flete_staging,
     (SELECT ROUND(SUM(freight_value),2)         FROM dwh.fact_ventas) AS flete_dwh;

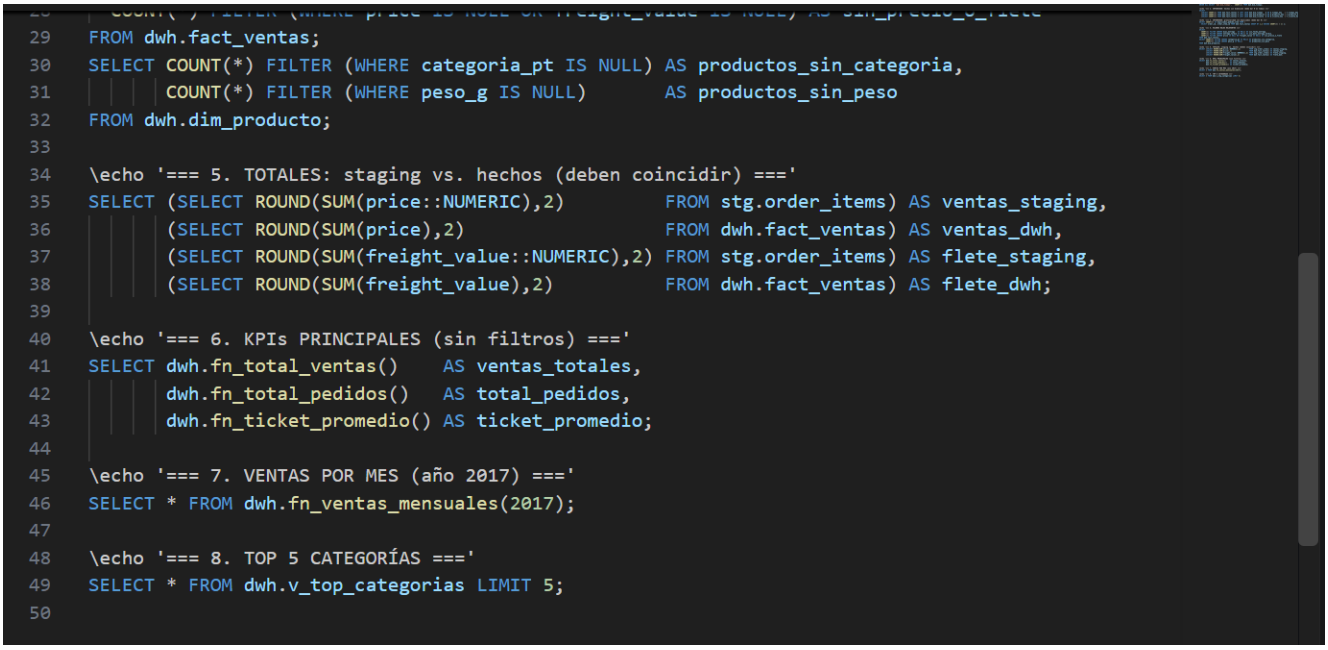
\echo '=== 6. KPIs PRINCIPALES (sin filtros) ==='
SELECT dwh.fn_total_ventas()    AS ventas_totales,
     dwh.fn_total_pedidos()   AS total_pedidos,
     dwh.fn_ticket_promedio() AS ticket_promedio;

\echo '=== 7. VENTAS POR MES (año 2017) ==='
SELECT * FROM dwh.fn_ventas_mensuales(2017);

\echo '=== 8. TOP 5 CATEGORÍAS ==='
SELECT * FROM dwh.v_top_categorias LIMIT 5;
```

Captura:





7. Vistas y funciones SQL iniciales

Archivo initdb/03_funciones_vistas.sql. Las funciones reciben tres filtros opcionales (año, estado del cliente y categoría); si un filtro es NULL, no se aplica. Todas excluyen los pedidos canceled y unavailable.

Objeto	Tipo	Alimenta
dwh.fn_total_ventas()	Función	Tarjeta: ventas totales
dwh.fn_total_pedidos()	Función	Tarjeta: total de pedidos
dwh.fn_ticket_promedio()	Función	Tarjeta: ticket promedio

Objeto	Tipo	Alimenta
dwh.fn_ventas_mensuales()	Función	Gráfico de líneas
dwh.fn_top_categorias()	Función	Gráfico de barras
dwh.fn_detalle_ventas()	Función	Tabla de detalle
dwh.v_top_categorias	Vista	Top de categorías sin filtros
dwh.v_detalle_ventas	Vista	Detalle de ventas sin filtros

Código

```
-- ================================================================
-- 03 · FUNCIONES Y VISTAS que alimentan el dashboard
-- (los pedidos 'canceled' y 'unavailable' no cuentan como venta)
-- ================================================================

-- KPI 1: Ventas totales
CREATE OR REPLACE FUNCTION dwh.fn_total_ventas(
   p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS NUMERIC AS $$
   SELECT COALESCE(SUM(f.price), 0)
   FROM dwh.fact_ventas f
   JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
   JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
   JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
   WHERE f.order_status NOT IN ('canceled','unavailable')
     AND (p_anio      IS NULL OR t.anio        = p_anio)
     AND (p_estado    IS NULL OR c.estado      = p_estado)
     AND (p_categoria IS NULL OR p.categoria_en = p_categoria);
$$ LANGUAGE sql STABLE;

-- KPI 2: Total de pedidos
CREATE OR REPLACE FUNCTION dwh.fn_total_pedidos(
   p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS BIGINT AS $$
   SELECT COUNT(DISTINCT f.order_id)
   FROM dwh.fact_ventas f
   JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
   JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
   JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
   WHERE f.order_status NOT IN ('canceled','unavailable')
     AND (p_anio      IS NULL OR t.anio        = p_anio)
     AND (p_estado    IS NULL OR c.estado      = p_estado)
     AND (p_categoria IS NULL OR p.categoria_en = p_categoria);
$$ LANGUAGE sql STABLE;

-- KPI 3: Ticket promedio = ventas / pedidos
CREATE OR REPLACE FUNCTION dwh.fn_ticket_promedio(
   p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS NUMERIC AS $$
   SELECT ROUND(dwh.fn_total_ventas(p_anio, p_estado, p_categoria)
               / NULLIF(dwh.fn_total_pedidos(p_anio, p_estado, p_categoria), 0), 2);
$$ LANGUAGE sql STABLE;
```

```
-- Gráfico de líneas: KPIs por mes
CREATE OR REPLACE FUNCTION dwh.fn_ventas_mensuales(
   p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS TABLE (
   periodo_anio SMALLINT, periodo_mes SMALLINT, mes_nombre VARCHAR(15),
   ventas_totales NUMERIC, total_pedidos BIGINT, ticket_promedio NUMERIC)
AS $$
   SELECT t.anio, t.mes, t.nombre_mes,
        SUM(f.price),
        COUNT(DISTINCT f.order_id),
        ROUND(SUM(f.price) / NULLIF(COUNT(DISTINCT f.order_id), 0), 2)
   FROM dwh.fact_ventas f
   JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
   JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
   JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
   WHERE f.order_status NOT IN ('canceled','unavailable')
     AND (p_anio      IS NULL OR t.anio        = p_anio)
     AND (p_estado    IS NULL OR c.estado      = p_estado)
     AND (p_categoria IS NULL OR p.categoria_en = p_categoria)
   GROUP BY t.anio, t.mes, t.nombre_mes
   ORDER BY t.anio, t.mes;
$$ LANGUAGE sql STABLE;

-- Gráfico de barras: top categorías
CREATE OR REPLACE VIEW dwh.v_top_categorias AS
SELECT p.categoria_en,
      SUM(f.price)             AS ventas_totales,
      COUNT(DISTINCT f.order_id) AS total_pedidos
FROM dwh.fact_ventas f
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
WHERE f.order_status NOT IN ('canceled','unavailable')
GROUP BY p.categoria_en
ORDER BY ventas_totales DESC;

-- Tabla de detalle
CREATE OR REPLACE VIEW dwh.v_detalle_ventas AS
SELECT f.order_id, t.fecha, p.categoria_en, c.estado AS estado_cliente,
      f.price, f.freight_value, f.order_status
FROM dwh.fact_ventas f
JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key;

-- Gráfico de barras con filtros: top categorías
CREATE OR REPLACE FUNCTION dwh.fn_top_categorias(
   p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL,
   p_categoria VARCHAR DEFAULT NULL, p_limite INT DEFAULT 10)
RETURNS TABLE (categoria VARCHAR(80), ventas_totales NUMERIC, total_pedidos BIGINT)
AS $$
   SELECT p.categoria_en, SUM(f.price), COUNT(DISTINCT f.order_id)
   FROM dwh.fact_ventas f
   JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
   JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
   JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
```

```
   WHERE f.order_status NOT IN ('canceled','unavailable')
     AND (p_anio      IS NULL OR t.anio        = p_anio)
     AND (p_estado    IS NULL OR c.estado      = p_estado)
     AND (p_categoria IS NULL OR p.categoria_en = p_categoria)
   GROUP BY p.categoria_en
   ORDER BY SUM(f.price) DESC
   LIMIT p_limite;
$$ LANGUAGE sql STABLE;

-- Tabla de detalle con filtros
CREATE OR REPLACE FUNCTION dwh.fn_detalle_ventas(
   p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL,
   p_categoria VARCHAR DEFAULT NULL, p_limite INT DEFAULT 20)
RETURNS TABLE (order_id VARCHAR(40), order_item_id SMALLINT, fecha DATE, categoria VARCHAR(80),
           estado_cliente CHAR(2), price NUMERIC, freight_value NUMERIC, order_status VARCHAR(20))
AS $$
   SELECT f.order_id, f.order_item_id, t.fecha, p.categoria_en, c.estado,
        f.price, f.freight_value, f.order_status
   FROM dwh.fact_ventas f
   JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
   JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
   JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
   WHERE f.order_status NOT IN ('canceled','unavailable')
     AND (p_anio      IS NULL OR t.anio        = p_anio)
     AND (p_estado    IS NULL OR c.estado      = p_estado)
     AND (p_categoria IS NULL OR p.categoria_en = p_categoria)
   ORDER BY t.fecha DESC, f.price DESC
   LIMIT p_limite;
$$ LANGUAGE sql STABLE;
```

Captura:

datamart_olist > initdb > 03_funciones_vistas.sql

```sql
-- ================================================================
-- 03 · FUNCIONES Y VISTAS que alimentan el dashboard
-- (los pedidos 'canceled' y 'unavailable' no cuentan como venta)
-- ================================================================

-- KPI 1: Ventas totales
CREATE OR REPLACE FUNCTION dwh.fn_total_ventas(
    p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS NUMERIC AS $$
    SELECT COALESCE(SUM(f.price), 0)
    FROM dwh.fact_ventas f
    JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
    JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
    JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
    WHERE f.order_status NOT IN ('canceled','unavailable')
      AND (p_anio      IS NULL OR t.anio         = p_anio)
      AND (p_estado    IS NULL OR c.estado       = p_estado)
      AND (p_categoria IS NULL OR p.categoria_en = p_categoria);
$$ LANGUAGE sql STABLE;

-- KPI 2: Total de pedidos
CREATE OR REPLACE FUNCTION dwh.fn_total_pedidos(
    p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS BIGINT AS $$
    SELECT COUNT(DISTINCT f.order_id)
    FROM dwh.fact_ventas f
    JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
    JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
    JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
```

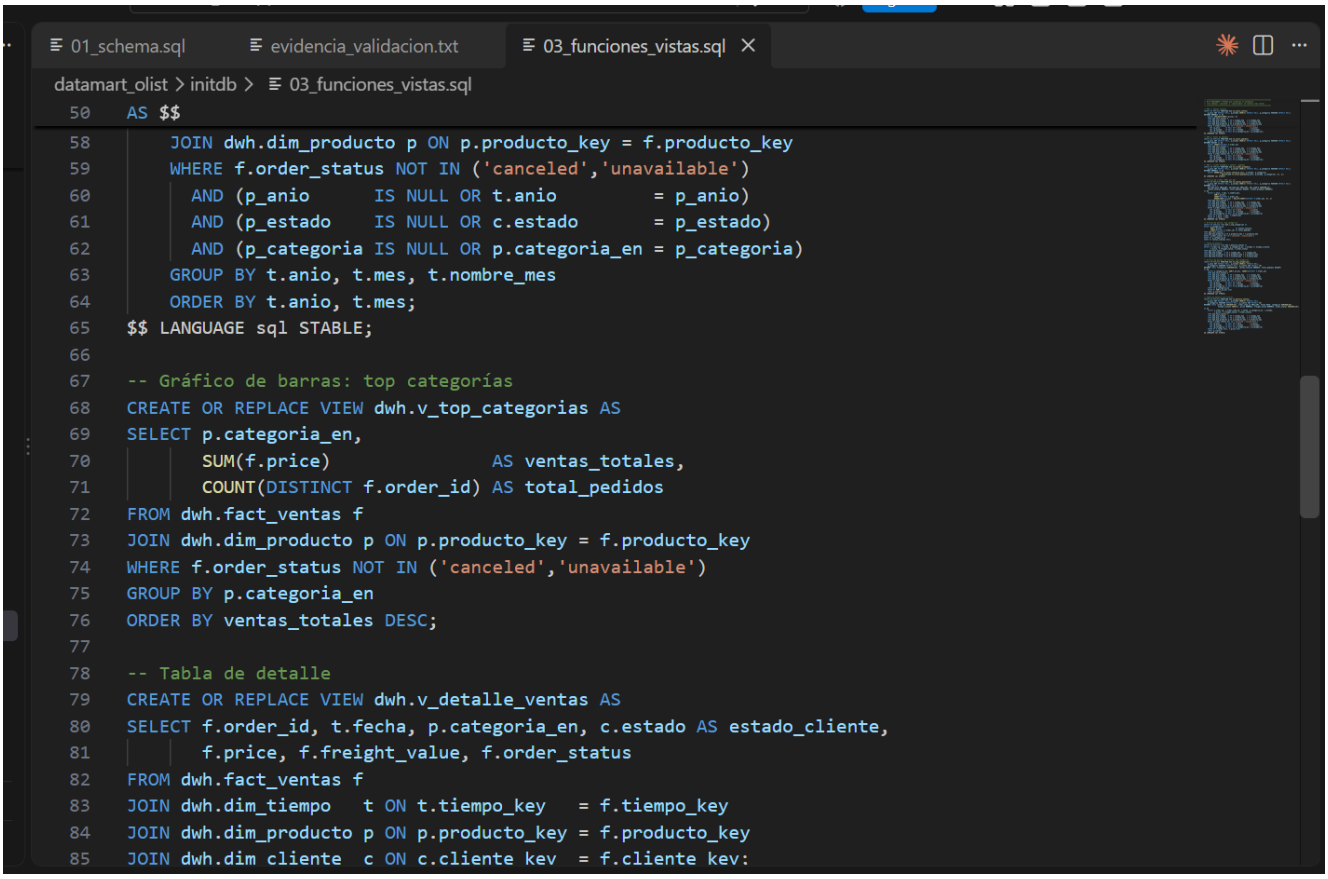
01_schema.sql | evidencia_validacion.txt | 03_funciones_vistas.sql

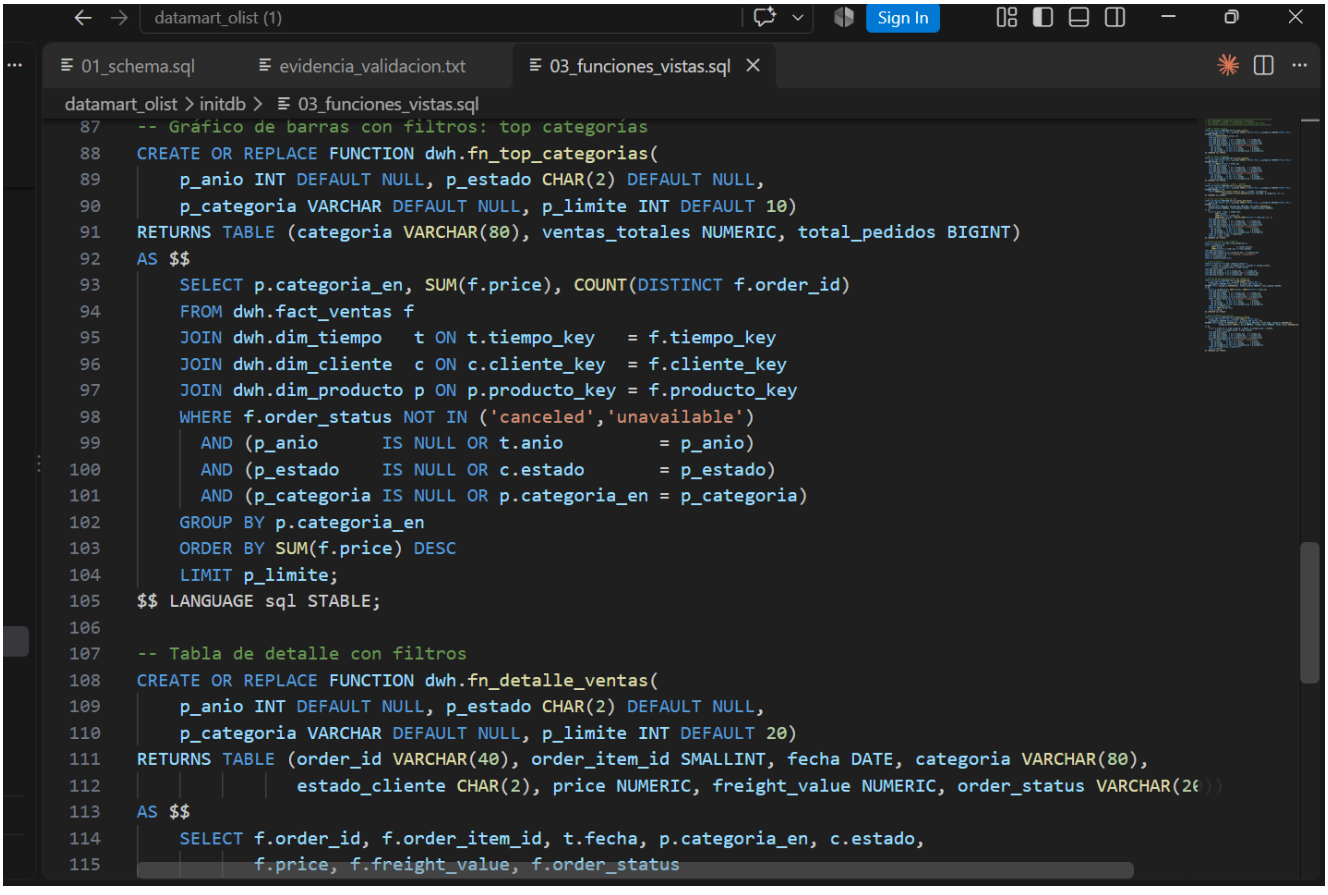
datamart_olist > initdb > 03_funciones_vistas.sql

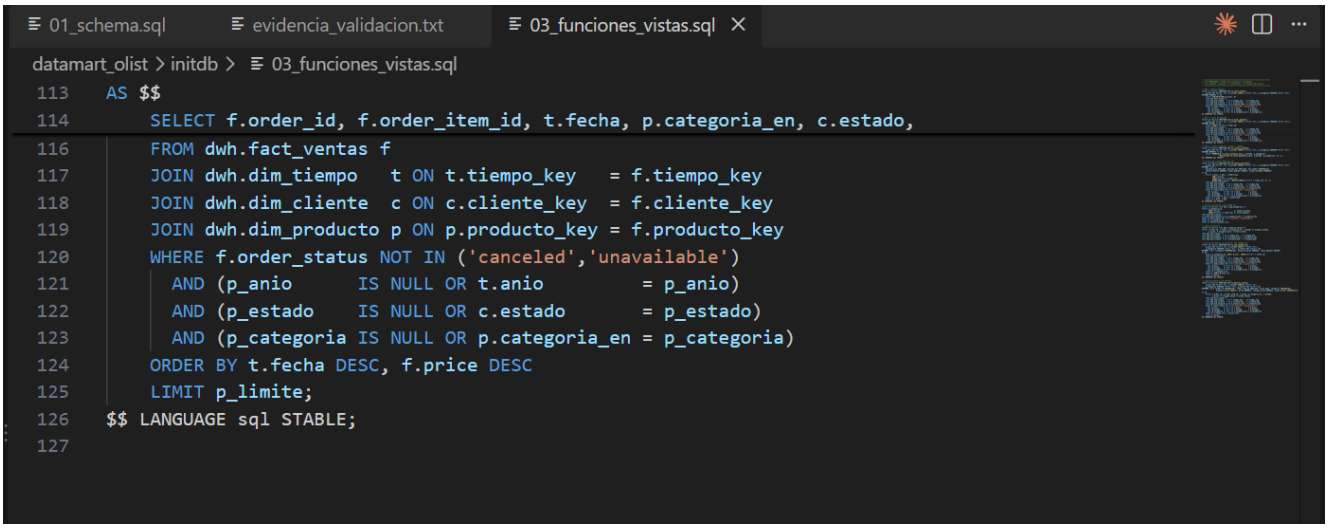
```sql
RETURNS BIGINT AS $$
    WHERE f.order_status NOT IN ('canceled','unavailable')
      AND (p_anio      IS NULL OR t.anio         = p_anio)
      AND (p_estado    IS NULL OR c.estado       = p_estado)
      AND (p_categoria IS NULL OR p.categoria_en = p_categoria);
$$ LANGUAGE sql STABLE;

-- KPI 3: Ticket promedio = ventas / pedidos
CREATE OR REPLACE FUNCTION dwh.fn_ticket_promedio(
    p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS NUMERIC AS $$
    SELECT ROUND(dwh.fn_total_ventas(p_anio, p_estado, p_categoria)
                 / NULLIF(dwh.fn_total_pedidos(p_anio, p_estado, p_categoria), 0), 2);
$$ LANGUAGE sql STABLE;

-- Gráfico de líneas: KPIs por mes
CREATE OR REPLACE FUNCTION dwh.fn_ventas_mensuales(
    p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL, p_categoria VARCHAR DEFAULT NULL)
RETURNS TABLE (
    periodo_anio SMALLINT, periodo_mes SMALLINT, mes_nombre VARCHAR(15),
    ventas_totales NUMERIC, total_pedidos BIGINT, ticket_promedio NUMERIC)
AS $$
    SELECT t.anio, t.mes, t.nombre_mes,
           SUM(f.price),
           COUNT(DISTINCT f.order_id),
           ROUND(SUM(f.price) / NULLIF(COUNT(DISTINCT f.order_id), 0), 2)
    FROM dwh.fact_ventas f
    JOIN dwh.dim_tiempo   t ON t.tiempo_key   = f.tiempo_key
    JOIN dwh.dim_cliente  c ON c.cliente_key  = f.cliente_key
```


01_schema.sql
evidencia_validacion.txt
03_funciones_vistas.sql
datamart_olist > initdb > 03_funciones_vistas.sql
AS $$
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
WHERE f.order_status NOT IN ('canceled','unavailable')
AND (p_anio IS NULL OR t.anio = p_anio)
AND (p_estado IS NULL OR c.estado = p_estado)
AND (p_categoria IS NULL OR p.categoria_en = p_categoria)
GROUP BY t.anio, t.mes, t.nombre_mes
ORDER BY t.anio, t.mes;
$$ LANGUAGE sql STABLE;
-- Gráfico de barras: top categorías
CREATE OR REPLACE VIEW dwh.v_top_categorias AS
SELECT p.categoria_en,
SUM(f.price) AS ventas_totales,
COUNT(DISTINCT f.order_id) AS total_pedidos
FROM dwh.fact_ventas f
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
WHERE f.order_status NOT IN ('canceled','unavailable')
GROUP BY p.categoria_en
ORDER BY ventas_totales DESC;
-- Tabla de detalle
CREATE OR REPLACE VIEW dwh.v_detalle_ventas AS
SELECT f.order_id, t.fecha, p.categoria_en, c.estado AS estado_cliente,
f.price, f.freight_value, f.order_status
FROM dwh.fact_ventas f
JOIN dwh.dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
JOIN dwh.dim cliente c ON c.cliente kev = f.cliente kev:


datamart_olist (1)
Sign In
01_schema.sql
evidencia_validacion.txt
03_funciones_vistas.sql
datamart_olist > initdb > 03_funciones_vistas.sql
-- Gráfico de barras con filtros: top categorías
CREATE OR REPLACE FUNCTION dwh.fn_top_categorias(
p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL,
p_categoria VARCHAR DEFAULT NULL, p_limite INT DEFAULT 10)
RETURNS TABLE (categoria VARCHAR(80), ventas_totales NUMERIC, total_pedidos BIGINT)
AS $$
SELECT p.categoria_en, SUM(f.price), COUNT(DISTINCT f.order_id)
FROM dwh.fact_ventas f
JOIN dwh.dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dwh.dim_cliente c ON c.cliente_key = f.cliente_key
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
WHERE f.order_status NOT IN ('canceled','unavailable')
AND (p_anio IS NULL OR t.anio = p_anio)
AND (p_estado IS NULL OR c.estado = p_estado)
AND (p_categoria IS NULL OR p.categoria_en = p_categoria)
GROUP BY p.categoria_en
ORDER BY SUM(f.price) DESC
LIMIT p_limite;
$$ LANGUAGE sql STABLE;
-- Tabla de detalle con filtros
CREATE OR REPLACE FUNCTION dwh.fn_detalle_ventas(
p_anio INT DEFAULT NULL, p_estado CHAR(2) DEFAULT NULL,
p_categoria VARCHAR DEFAULT NULL, p_limite INT DEFAULT 20)
RETURNS TABLE (order_id VARCHAR(40), order_item_id SMALLINT, fecha DATE, categoria VARCHAR(80),
estado_cliente CHAR(2), price NUMERIC, freight_value NUMERIC, order_status VARCHAR(20))
AS $$
SELECT f.order_id, f.order_item_id, t.fecha, p.categoria_en, c.estado,
f.price, f.freight_value, f.order_status


01_schema.sql
evidencia_validacion.txt
03_funciones_vistas.sql
datamart_olist > initdb > 03_funciones_vistas.sql
AS $$
SELECT f.order_id, f.order_item_id, t.fecha, p.categoria_en, c.estado,
FROM dwh.fact_ventas f
JOIN dwh.dim_tiempo t ON t.tiempo_key = f.tiempo_key
JOIN dwh.dim_cliente c ON c.cliente_key = f.cliente_key
JOIN dwh.dim_producto p ON p.producto_key = f.producto_key
WHERE f.order_status NOT IN ('canceled','unavailable')
AND (p_anio IS NULL OR t.anio = p_anio)
AND (p_estado IS NULL OR c.estado = p_estado)
AND (p_categoria IS NULL OR p.categoria_en = p_categoria)
ORDER BY t.fecha DESC, f.price DESC
LIMIT p_limite;
$$ LANGUAGE sql STABLE;

8. Mockup del dashboard

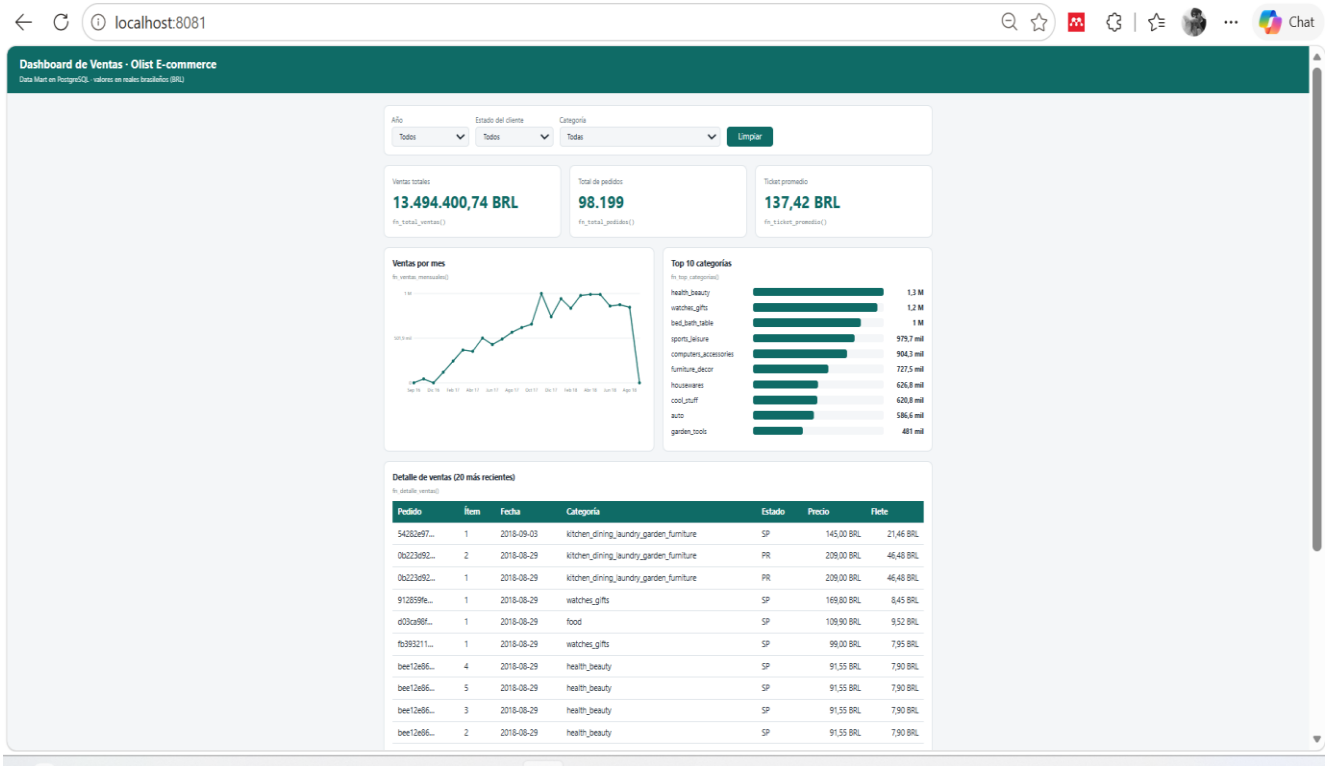


Figura 2. Mockup del dashboard con datos reales del año 2017.

El mockup incluye los elementos mínimos solicitados:

- **3 KPI:** ventas totales, total de pedidos y ticket promedio.
- **Filtros:** año, estado del cliente y categoría de producto, más un botón para limpiarlos.
- **2 gráficos:** líneas con las ventas por mes y barras con el top de categorías.
- **Tabla de detalle:** pedido, fecha, categoría, estado, precio y flete.
- **Organización:** filtros arriba, KPI en una fila, gráficos en paralelo y detalle al final.

El dashboard está implementado y funcional en http://localhost:8081 (carpeta app/): una API en Flask consulta las funciones SQL y la página actualiza KPI, gráficos y tabla al cambiar cualquier filtro.

9. Matriz de trazabilidad Mockup → KPI → Filtros → SQL

Cada componente del dashboard se relaciona con la información almacenada en PostgreSQL.

Componente	Pregunta / KPI	Filtros	Origen de datos	Objeto SQL
Tarjeta 1	¿Cuánto se vendió en total?	Año, estado, categoría	fact_ventas + dim_tiempo + dim_cliente + dim_producto	fn_total_ventas()
Tarjeta 2	¿Cuántos pedidos hubo?	Año, estado, categoría	fact_ventas + dimensiones	fn_total_pedidos()
Tarjeta 3	¿Cuál es el valor promedio por pedido?	Año, estado, categoría	fact_ventas + dimensiones	fn_ticket_promedio()
Gráfico de líneas	¿Cómo evolucionan las ventas por mes?	Año, estado, categoría	fact_ventas + dimensiones	fn_ventas_mensuales()
Gráfico de barras	¿Qué categorías venden más?	Año, estado, categoría	fact_ventas + dim_producto + dimensiones	fn_top_categorias()
Tabla de detalle	¿Qué ítems componen las ventas?	Año, estado, categoría	fact_ventas + dimensiones	fn_detalle_ventas()

Pregunta que debe poder responder al finalizar esta fase:
¿Cómo voy a almacenar, consultar y presentar la información?

Almacenar

Los datos del dataset Olist (e-commerce de Brasil) se guardan en **PostgreSQL**, dentro de un **modelo dimensional en estrella**:

- **Una tabla de hechos**, fact_ventas, donde cada fila es un ítem vendido dentro de un pedido. Guarda las medidas: precio, flete, total del ítem, días de entrega y si la entrega fue tardía.
- **Tres dimensiones**: dim_tiempo (año, trimestre, mes), dim_producto (categoría y peso) y dim_cliente (ciudad y estado).
- **Claves primarias y foráneas** que conectan los hechos con las dimensiones, además de restricciones CHECK y UNIQUE para proteger la calidad de los datos.

Antes de llegar al modelo, los datos se limpian: se traducen las categorías, los 610 productos sin categoría pasan a sin_categoria, se usa el cliente real (customer_unique_id) y se respetan los nulos de entrega. El proceso es CSV → staging (stg) → modelo (dwh) y queda cargado con 112.650 registros.

Consultar

La información se consulta con **vistas y funciones SQL** que reciben tres filtros opcionales: año, estado del cliente y categoría.

- fn_total_ventas(), fn_total_pedidos() y fn_ticket_promedio() calculan los 3 KPI.
- fn_ventas_mensuales() y fn_top_categorias() alimentan los gráficos.
- fn_detalle_ventas() alimenta la tabla de detalle.

Con ellas, el cálculo queda en la base de datos y cada componente del dashboard tiene un origen claro y trazable. Las consultas de validación comprueban que todo esté bien: los registros cargados, las relaciones sin huérfanos, los nulos y que los totales coincidan con el origen.

Presentar

La información se presenta en un **dashboard web** con:

- **3 KPI:** ventas totales (R$ 13,49 M), total de pedidos (98.199) y ticket promedio (R$ 137,42).
- **Filtros** de año, estado y categoría.
- **2 gráficos:** ventas por mes (líneas) y top de categorías (barras).
- **Tabla de detalle** con los ítems más recientes.

Una API en Flask conecta la página con las funciones SQL, de modo que al cambiar un filtro se actualiza todo. La **matriz de trazabilidad** documenta la relación Mockup → KPI → Filtros → SQL de cada componente.